

Informe de Pruebas Unitarias y Aseguramiento de la Calidad

Integrantes:

- Sebastián Carrera

Asignatura: Fullstack III

Informe de Pruebas Unitarias y Aseguramiento de la Calidad.....	0
Introducción.....	2
Diseño de la arquitectura de microservicios.....	3
Decisiones en el desarrollo de componentes backend y frontend.....	5
Aplicación de patrones de diseño.....	6
Estrategia de branching y gestión de versiones.....	7
Integración de frontend, backend y bases de datos.....	7
Pruebas unitarias y aseguramiento de la calidad.....	8
Conclusión.....	11

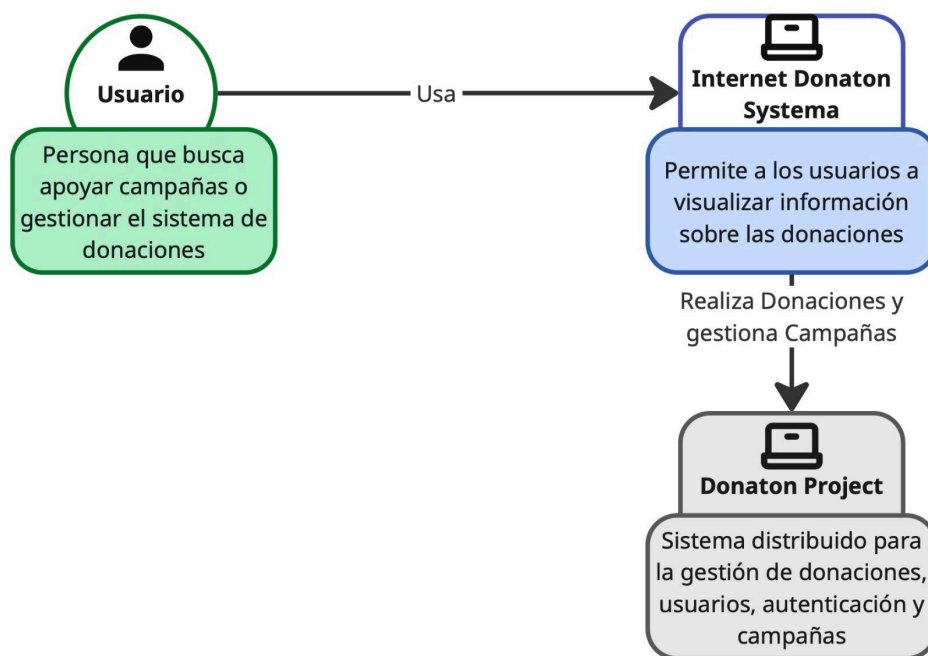
Introducción

Donaton es una plataforma digital orientada a gestionar y coordinar ayuda humanitaria durante emergencias y desastres naturales. El problema abordado corresponde a la falta de trazabilidad de las donaciones, el uso de registros manuales y la dificultad para coordinar campañas, usuarios y recursos entre diferentes participantes.

La solución desarrollada integra un frontend web, un API Gateway, un Backend for Frontend y microservicios especializados en usuarios, autenticación, campañas y donaciones. Los componentes se comunican mediante servicios REST, utilizan persistencia independiente cuando corresponde y pueden ejecutarse con Docker o Kubernetes.

El desarrollo se realizó de manera incremental, incorporando funcionalidades, documentación de API, pruebas automatizadas, observabilidad y mecanismos de despliegue. Este informe analiza retrospectivamente las principales decisiones tomadas, sus resultados y las mejoras que podrían incorporarse en una futura evolución del sistema.

Diagrama C1: Diagrama de Contexto



Diseño de la arquitectura de microservicios

La arquitectura fue diseñada para separar las principales responsabilidades de negocio. En lugar de construir una aplicación monolítica, se dividió la solución en servicios especializados:

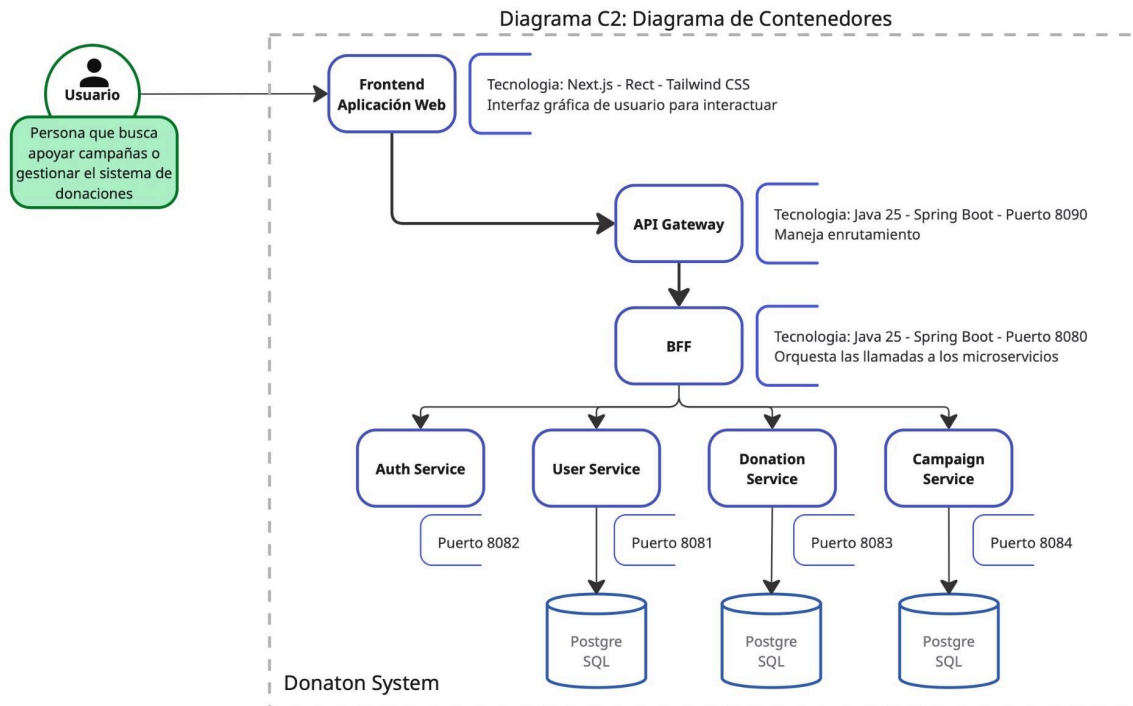
- **ms-user:** administra usuarios y datos personales.
- **ms-auth:** valida credenciales y genera tokens JWT.
- **ms-campaign:** gestiona campañas solidarias.
- **ms-donation:** registra y consulta donaciones.
- **BFF:** coordina las operaciones requeridas por el frontend.
- **API Gateway:** centraliza el acceso externo.
- **Frontend:** proporciona la interfaz utilizada por los usuarios.

El flujo general comienza en el frontend. Las solicitudes son enviadas al API Gateway, que las dirige al BFF. Este último determina qué microservicios deben intervenir y coordina sus respuestas. De esta forma, la interfaz no necesita conocer directamente la ubicación ni la implementación de cada servicio.

Esta distribución responde a los requerimientos del cliente porque permite gestionar separadamente usuarios, campañas y donaciones, manteniendo trazabilidad sobre cada operación. También facilita la evolución del sistema, ya que un servicio puede modificarse sin intervenir directamente en todos los demás componentes.

Los principales patrones arquitectónicos utilizados fueron **Microservices**, **API Gateway** y **Backend for Frontend**. La arquitectura de microservicios permitió separar capacidades de negocio; el Gateway proporcionó un punto de entrada único; y el BFF adaptó las operaciones del backend a las necesidades de la interfaz.

En materia de seguridad, las contraseñas se almacenan codificadas y la autenticación se realiza mediante tokens JWT. Las credenciales de las bases de datos no se incluyen directamente en los manifiestos Kubernetes versionados, sino que son proporcionadas mediante un Secret local. Las direcciones internas de los servicios se administran mediante un ConfigMap.



Desde la perspectiva de privacidad y responsabilidad, el sistema almacena solamente la información requerida para identificar usuarios, gestionar campañas y registrar donaciones. No obstante, una versión productiva debería reforzar la protección de sesión mediante cookies **HttpOnly**, cifrado de datos sensibles, expiración de tokens y políticas formales de control de acceso.

La arquitectura también considera escalabilidad. Cada servicio se ejecuta en un Deployment independiente de Kubernetes, por lo que podría aumentar sus réplicas según su demanda. Aunque en el prototipo se utiliza una réplica por componente, la estructura permite aplicar posteriormente autoscaling.

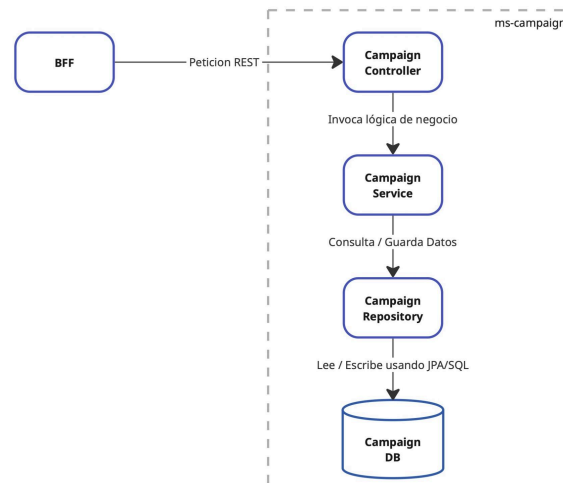
Como medida de sostenibilidad técnica, se definieron límites de CPU y memoria, evitando un consumo descontrolado de recursos. Además, el script de administración permite escalar los Deployments a cero réplicas cuando el entorno no está siendo utilizado.

Decisiones en el desarrollo de componentes backend y frontend

Los componentes backend fueron desarrollados con Java y Spring Boot. Esta elección permitió utilizar un entorno común para controladores REST, servicios, persistencia, validaciones, seguridad y pruebas. Gradle fue utilizado para administrar dependencias, compilar el código y generar los archivos ejecutables.

La estructura interna de los microservicios sigue una organización por capas: controlador, servicio, repositorio, entidad, DTO, excepciones y configuración. Esta distribución evita concentrar toda la lógica en una sola clase y facilita la modificación o prueba de cada responsabilidad.

Diagrama C3: Diagrama de Componentes



Spring Data JPA permitió implementar los repositorios con menos código repetitivo. Operaciones como guardar, listar, buscar o eliminar entidades se realizan mediante interfaces que extienden `JpaRepository`. Flyway fue utilizado para versionar la estructura de las bases de datos mediante scripts SQL.

El frontend fue desarrollado con Next.js, React y TypeScript. Next.js facilitó la organización de rutas públicas y protegidas, mientras que React permitió construir componentes reutilizables. TypeScript aportó tipado estático y ayudó a detectar errores antes de ejecutar la aplicación.

La interfaz se organizó en páginas, componentes, hooks, servicios, tipos y utilidades. Los servicios del frontend encapsulan la comunicación con el BFF, evitando llamadas HTTP distribuidas sin una estructura común. Las funciones de sesión y comunicación también fueron centralizadas para reducir duplicación.

Una decisión importante fue incorporar un BFF. Por ejemplo, durante el inicio de sesión, el BFF consulta los datos de autenticación del usuario y después solicita al servicio de autenticación la generación del token. En la creación de donaciones, valida la sesión, obtiene el usuario autenticado y asigna su identificador como donante. Esto evita que el frontend coordine directamente varios microservicios.

Entre los beneficios obtenidos destacan la modularidad, la claridad de responsabilidades y la posibilidad de probar cada capa de manera independiente. Sin embargo, el desarrollo también evidenció oportunidades de mejora. Algunos contratos entre servicios podrían estar más estandarizados, especialmente las respuestas de error. También sería conveniente reemplazar progresivamente clases con getters manuales por records o estructuras homogéneas cuando resulte apropiado.

En el frontend, la cobertura de pruebas de componentes visuales todavía puede ampliarse. Asimismo, en una versión productiva se recomienda administrar la autenticación mediante cookies seguras y no únicamente mediante almacenamiento de sesión en el navegador.

Aplicación de patrones de diseño

El patrón **Repository** se implementó mediante Spring Data JPA en los microservicios con persistencia. Los servicios acceden a interfaces como **UserRepository**, **CampaignRepository** y **DonationRepository**, sin depender directamente de consultas SQL específicas. Esto redujo el acoplamiento entre la lógica de negocio y la base de datos.

La arquitectura por capas complementa este patrón. Los controladores reciben las solicitudes HTTP, los servicios ejecutan las reglas de negocio y los repositorios administran los datos. Los DTO permiten definir contratos de entrada y salida sin exponer directamente las entidades persistentes.

También se aplicó **Dependency Injection**, administrada por Spring. Los servicios reciben sus repositorios, codificadores de contraseña y otras dependencias mediante constructores. Esto facilitó las pruebas unitarias, ya que fue posible reemplazar las dependencias reales por mocks.

En el frontend se utilizó una abstracción similar mediante módulos de servicio. **authService**, **userService**, **campaignService** y **donationService** encapsulan las operaciones asociadas a cada dominio, mientras que las funciones comunes de comunicación se concentran en un único módulo.

Los patrones aplicados contribuyeron a la modularidad y reutilización del código. El Repository Pattern evitó repetir operaciones de persistencia; los servicios del frontend centralizaron las llamadas HTTP; y el BFF reutilizó una función común para comunicarse con los microservicios.

Durante el diseño inicial también se consideraron patrones como Circuit Breaker y Factory Method. Sin embargo, no fueron implementados formalmente debido a la priorización de la integración, estabilidad y pruebas dentro del plazo disponible. Como mejora futura, un Circuit Breaker en el BFF permitiría controlar fallos temporales de los microservicios mediante timeouts, reintentos y respuestas alternativas.

Estrategia de branching y gestión de versiones

El proyecto fue versionado mediante Git y almacenado en GitHub. El repositorio contiene el frontend, los microservicios, el BFF, el API Gateway, los manifiestos Kubernetes, los Dockerfiles, los scripts de automatización, la documentación OpenAPI y las evidencias de pruebas.

Durante el desarrollo se utilizaron ramas para separar avances funcionales y técnicos. Se trabajó en etapas relacionadas con frontend, autenticación, campañas, donaciones, Docker, Kubernetes, monitoreo y pruebas. Esto permitió realizar cambios sin intervenir inmediatamente en la versión principal.

Los commits se realizaron de manera incremental, asociados a funcionalidades o correcciones específicas. Esta práctica facilitó la identificación de cambios, la recuperación de versiones anteriores y la organización del trabajo realizado.

La estrategia fue útil para mantener estabilidad y disminuir el riesgo de perder avances. Sin embargo, no se aplicó un flujo de trabajo completamente formal. Como mejora futura se recomienda utilizar una estructura definida con ramas `main`, `develop` y `feature/*`, además de pull requests, revisión de código y protección de la rama principal.

También sería recomendable incorporar convenciones como Conventional Commits y crear releases para identificar versiones estables. Estas prácticas mejorarían la coordinación en un equipo de mayor tamaño y permitirían automatizar validaciones antes de integrar cambios.

Integración de frontend, backend y bases de datos

La integración sigue el flujo:

Frontend → API Gateway → BFF → Microservicios → Bases de datos

El frontend envía sus solicitudes a rutas bajo `/api`. El API Gateway centraliza el acceso y dirige las peticiones al BFF. El BFF coordina las llamadas necesarias hacia usuarios, autenticación, campañas o donaciones.

Los microservicios de usuarios, campañas y donaciones utilizan bases de datos PostgreSQL independientes alojadas en Neon. Esta separación permite que cada servicio administre su información sin acceder directamente a las tablas de otro componente.

JPA se utiliza para mapear las entidades Java con las tablas de PostgreSQL. Flyway administra las migraciones y mantiene scripts versionados para crear tablas e insertar datos iniciales. Esto proporciona una forma reproducible de preparar la persistencia.

Cada componente cuenta con un Dockerfile. Docker Compose fue utilizado para ejecutar todos los servicios dentro de una red compartida, utilizando los nombres de los contenedores como direcciones internas.

Posteriormente, la solución fue desplegada en Kubernetes mediante Docker Desktop. Se configuraron un Namespace, un ConfigMap, un Secret local, Deployments y Services. Los servicios internos utilizan **ClusterIP**, mientras que el frontend dispone de un **NodePort**.

Se desarrolló el script **donaton.sh** para automatizar la ejecución. Este valida herramientas, manifiestos e imágenes; aplica los recursos Kubernetes; espera que los Deployments estén disponibles; crea los port-forward; y verifica los endpoints de salud. También permite consultar el estado, detener los accesos locales o suspender los Pods.

Los principales desafíos de integración fueron la configuración de direcciones diferentes para ejecución local, Docker y Kubernetes; la administración de puertos; el manejo de CORS; las credenciales externas; y la recuperación de los port-forward después de reiniciar el equipo.

Estos problemas fueron resueltos mediante variables de entorno, ConfigMaps, Services de Kubernetes y automatización. El resultado fue una solución que puede iniciarse de forma repetible y cuya comunicación fue validada desde la interfaz hasta la persistencia.

Pruebas unitarias y aseguramiento de la calidad

Las pruebas automatizadas fueron implementadas tanto en el frontend como en el backend. Se utilizaron Vitest y Testing Library en la interfaz, y JUnit, Mockito, Spring Boot Test, H2 y JaCoCo en los servicios Java.

En total se ejecutaron **163 pruebas sin fallos**:

Componente	Pruebas
Frontend	42
ms-user	16

ms-auth	15
ms-campaign	19
ms-donation	19
BFF	39
API Gateway	13
Total	163

En el frontend, la cobertura global alcanzó aproximadamente un 25 %. Este porcentaje se explica porque existen numerosos componentes visuales y hooks todavía no probados. Sin embargo, los módulos críticos de sesión, comunicación con el BFF y servicios alcanzaron 100 % de cobertura.

En el backend, las capas principales presentaron una cobertura alta. Los servicios de autenticación, campañas, donaciones y BFF alcanzaron 100 % en las clases consideradas. El microservicio de usuarios obtuvo aproximadamente 84 % global y 100 % en su capa de servicio. El API Gateway logró 100 % en su configuración propia y además incorporó pruebas reales de enrutamiento.

Las pruebas permitieron validar reglas de negocio, persistencia, controladores, autenticación, manejo de errores y comunicación. En el BFF se probaron flujos como registro, login, validación de token, consulta de perfil, creación de donaciones y operaciones de campañas. En el Gateway se simuló un servicio downstream para verificar rutas, respuestas 404 y CORS.

Uno de los principales desafíos fue aislar los componentes de dependencias externas. Para evitar utilizar las bases de datos reales durante las pruebas, se configuró H2. En el Gateway fue necesario crear un servidor HTTP simulado y proporcionar dinámicamente sus propiedades.

También se enfrentaron incompatibilidades de dependencias, ajustes de Mockito, errores de contexto de Spring y diferencias entre cobertura funcional y cobertura de líneas. La resolución de estos problemas permitió obtener pruebas reproducibles y reportes de evidencia.

Las pruebas contribuyeron a la estabilidad porque detectaron errores antes de volver a construir las imágenes o desplegar los componentes. Además, facilitan futuras modificaciones, ya que permiten verificar rápidamente si una funcionalidad existente fue afectada.

Como mejora futura se propone aumentar las pruebas de componentes visuales, incorporar pruebas end-to-end y establecer umbrales mínimos de cobertura. También sería conveniente automatizar su ejecución mediante integración continua.

Conclusión

El proyecto Donaton permitió construir una plataforma funcional basada en microservicios para gestionar usuarios, autenticación, campañas y donaciones. La combinación de API Gateway, BFF, servicios especializados y persistencia independiente permitió separar responsabilidades y mantener una comunicación organizada.

Las decisiones adoptadas favorecieron la modularidad, mantenibilidad y capacidad de despliegue. Docker y Kubernetes permitieron ejecutar la solución de manera consistente, mientras que OpenAPI, GlitchTip y los scripts desarrollados aportaron documentación, observabilidad y automatización.

Las 163 pruebas aprobadas entregan evidencia de la calidad alcanzada, especialmente en las capas backend y en los módulos críticos del frontend. Aun cuando existen oportunidades de mejora en resiliencia, seguridad, cobertura visual y automatización continua, la versión obtenida cumple con el objetivo de presentar una solución integrada, desplegable y técnicamente justificable.

La principal conclusión del proceso es que una arquitectura de microservicios requiere más que separar el código en proyectos independientes. También exige administrar contratos, comunicación, datos, seguridad, despliegue y pruebas. La integración de estos elementos permitió transformar los componentes desarrollados durante el semestre en una solución coherente y demostrable.